

Practical 1a. Introduction to Time Series Analysis with R

R is one of the current standards for statistical data analysis. In this practical we will use R for classical analysis of time series. I suggest the use of RStudio as a development framework.

In this practical we will cover the following topics:

- Introduction to RStudio.
- Graphical representation of time series.
- Time Series Transformation.
- Linear regression. Linear trend estimation.
- Seasonal component estimation.
- Filtering.
- Differentiation.

1. Importing data files with RStudio & preliminary manipulations.

RStudio can import excel files “name.xlsx” with the package ‘readxl’. You may have to install it by using the command t.

Once installed, import the content of the file “Air_Passengers.xlsx”.

To this end, type in the following commands:

- `x<-AirPassengers$Passengers`
- `t<-AirPassengers$Date`

to create vectors x and t.

Install now the package “tseries”. (`install.packages('tseries')`) After doing this, type in:

- `xt<-ts(x,start=1949,frequency=12)`
- `xt`

and check out the result.

The function “ls()” will yield a list of defined objects. The function “rm(name of an object)” allows you to remove an object.

There is a useful ‘help’ function.

If you want to spare yourself the data import from the excel file, you may also access the AirPassengers dataset directly, as it is contained in the set of already available datasets under that precise name, and with a time series format. It can be recalled with the command `data(AirPassengers)`

Create the time series by:

- `AP<- Air_Passengers`
- `AP`

The instruction 'class(AP)' shows the type of file we have at hand. The instruction summary(AP) provides the summary statistics of the time series:

We have two identical objects, a time series: xt or AP. We will only use xt from now onwards.

2. Graphical representation.

With the following instructions, we have different option for graphical representation of data:

- plot(xt, main="Airline Passengers", xlab="time", ylab="passengers", type="o")
- plot(t, x, main="Airline Passengers", xlab="time", ylab="passengers", type="l")

Check the help for the "plot" function, in particular the different options for "type".

Another interesting option is to use aggregate and cycle to make a yearly summary. Try:

- layout (1:2)
- plot(aggregate(xt))
- boxplot(xt~cycle(xt))

and see the result. The command layout facilitates the presentation of joint graphics. Check the help of this command.

3. Transforming a time series.

Please type in:

- y<-log(x)
- y
- plot (t, y, xlab="time", ylab="passengers", type="o")

The vector y contains the series of logarithms of the monthly number of passengers. Note that in the graphical representation the apparent non-stationarity of the variance has disappeared.

R is very powerful representing functions. For example:

- t<-1:500
- c<-2*cos(2*pi*t/50+0.6*pi)
- plot(t, c, type="l")

4. Estimating a linear trend.

We have two objects defined, t and x. To obtain the *best linear approximation to data* we can do as described next:

- t<-1:144
- line=lm(y~t)
- summary(line)
- plot(t,y,type="o",xlab="time",ylab="passengers");
- abline(line)

To extract the trend to the logarithmic data we can do:

- y<-log(x)
- lm(y~t)
- trend<-0.01005*t+4.81367
- trend

- `z<-y-trend`
- `z`
- `plot.ts(z)`

Where `z` is the variable containing the residuals of the time series after log transforming and extracting the linear trend.

5. Estimating the seasonal component.

To eliminate seasonality of `z` we can do:

- `dim(z)<-c(12,12)`
- `z`

We have converted the series in a matrix where rows are data of the same month. Now we compute monthly row means, the *global mean* and the *seasonal component*.

- `e<-apply(z,1,mean)`
- `ee<-mean(e)`
- `s<-e-ee`
- `plot (s,type="o")`

We have now captured the seasonal component. Now we do:

- `est<-array(s,144)`
- `est`
- `plot (est,type="o")`

Ultimately:

- `dim(z)<-144`
- `u<-z-est`
- `u`
- `plot (u,type="o")`

Note that series `u` is a series without trend nor seasonality. What can we do with it?

6. Filtering

Given `x`, the following instruction applies a *moving average* of order $q=1$ to the series:

- `v<-filter(x, sides=2,rep(1,3)/3)`

The function creates the vector $(1,1,1)$. Note that the first and the last values in `v` series are lost. Increase progressively 3 by 5 and 7 and see the graphics.

7. Differentiation.

Given `x` and `d`

- `y<-log(x)`
- `dy<-diff(y)`
- `dy`
- `plot (dy,type="o")`

This is the graph of the first order difference. To obtain differences of order 12 we can do:

➤ `de<-diff(y, lag=12)`

If we do

➤ `de<-diff(y,12,2)`

we are doing second differences of order 12.

Do

➤ `y<-log(x)`
➤ `dy12<-diff(y, lag=12)`
➤ `plot(dy12, xlab="time", ylab="passengers", type="o")`
➤ `ddy12<-diff(dy12)`
➤ `plot(ddy12, xlab="time", ylab="passengers", type="o")`

The last instruction shows the series of residuals after applying `nabla12` and `nabla`.

8. Decompositions

Given a time series like `x_t`, we can decompose directly, using a moving average only. Next is an example:

➤ `plot(decompose(xt))`

and a more detailed construction:

➤ `dec<-decompose(xt, type="additive")`
➤ `plot(dec)`
➤ `trend<-dec$trend`
➤ `seasonal<-dec$seasonal`
➤ `random<-dec$random`
➤ `layout(1:3)`
➤ `plot(trend)`
➤ `plot(seasonal)`
➤ `plot(random)`

Practical 1b. Simulation of Time Series in R

1. IID and Gaussian white noises

Type down the following commands:

- `n<-1000`
- `z1<-rnorm(n)`
- `z1`

With this, we have obtained a sample of n data points from with a standard normal law. In Time Series language, this is standard *Gaussian White Noise*. We can also do this directly:

- `z1<-rnorm(100)`
- `z1`

although the former option provides more programming flexibility.

If we type:

- `mu=5`
- `sigma=2`
- `z2<-rnorm(n,mean=mu,sd=sigma)`
- `z2`

we would be simulating a sample set of a normal random variable with mean 5 and standard deviation 2, or, in other words, a Gaussian White Noise with mean 5 and standard deviation 2.

Try as well:

- `m<-1`
- `set.seed(m)`
- `w<-rnorm(1000)`
- `w`

and choose values for m , different than 1. Note that by default, R chooses a random seed. If we fix the seed, we will obtain the same sample repeatedly. We can plot the graphics of our samples using for example:

- `plot(z2,type="l")`

You may improve the plot using instructions as `xlab`, `ylab`, (to add labels) etc. Please, refer to the help to access the full instruction description.

If we type:

- `hist(z1,freq=F)`

we obtain the histogram of a standard normal sample. See the help of the instruction `hist`. What happens if we remove the parameter `freq=F`? Repeat the same exercise for different values of n , μ and σ and check out what happens.

To compare how samples differs from their underlying population do:

- `n<-1000`
- `x<-seq(-4,4, length=1000)`

- `hist(rnorm(n), prob=T);`
- `points(x,dnorm(x), type="l")`

Try with a bigger n. It is also relevant to compute the Correlogram:

- `N=10000`
- `mu=5`
- `sigma=2`
- `z<-rnorm(n,mean=mu,sd=sigma)`
- `z`
- `acf(z)`

Observe that this is a correlogram of an IID-noise.

B) Random walk:

With the following commands:

- `n<-1000`
- `z<-rnorm(n)`
- `z`
- `x<-cumsum(z)`
- `plot(x,type="o")`

we will obtain a trajectory of a standard random walk. Test the same with other Gaussian white noises with different means and variances.

An alternative to do the same is the following:

- `n<-10000`
- `w<-rnorm(n)`
- `x<-w`
- `for (t in 2:n) x[t]<-x[t-1]+w[t]`
- `plot(x,type="l")`

Finally, we can observe that its first difference is IID noise:

- `acf(diff(x))`